ON NEUROMORPHIC COMPUTING

WITH SPIKING NEURAL NETWORKS

08.06.2026

INTRODUCTION AND MOTIVATION

- Deep learning has exceeded human performance in some domains and specific problems — AlphaFold, GPT, AlphaGo [1], [2]
- Yet training state-of-the-art models consumes vast energy and is rapidly approaching physical and economic limits [3], [4], [5], [6], [7]
- The human brain operates on 20 W and still outperforms AI on adaptation, few-shot learning, and common-sense reasoning [8]

Let’s draw more inspiration from biological intelligence → **neuromorphic computing**

Key takeaway lessons from the brain and what we should base our neuromorphic architecture on:

- **Sparse:** Only sends and computes when needed — silence is free
- **Event driven:** Highly efficient neuron communication via binary spikes; information encoded in timing, not magnitude
- **Local:** Neurons hold their own weights; compute and memory are co-located, physically in the same place (eliminating the von Neumann bottleneck)

RESEARCH OBJECTIVES

Sparse Efficient Computing: Investigate whether biologically inspired, event-driven algorithms reduce the computational footprint of visual classification when simulated on standard hardware.

Neuron Model Evaluation: Identify which temporal integration dynamics are compatible with Time-To-First-Spike (TTFS) rank-order decoding across systematic threshold regimes.

Inference Via Weight Transfer: Quantify the accuracy penalty of zero-shot Artificial-Neural-Network (ANN) to Spiking-Neural-Network (SNN) weight transfer under TTFS encoding, isolating the cost of transitioning to event-driven integration.

Native Unsupervised Learning: Determine if local Spike-Time-Dependent-Plasticity (STDP) can autonomously extract meaningful geometric features from visual input without global error signals.

First let's look at how the brain works...

NEURON STRUCTURE & FUNCTION

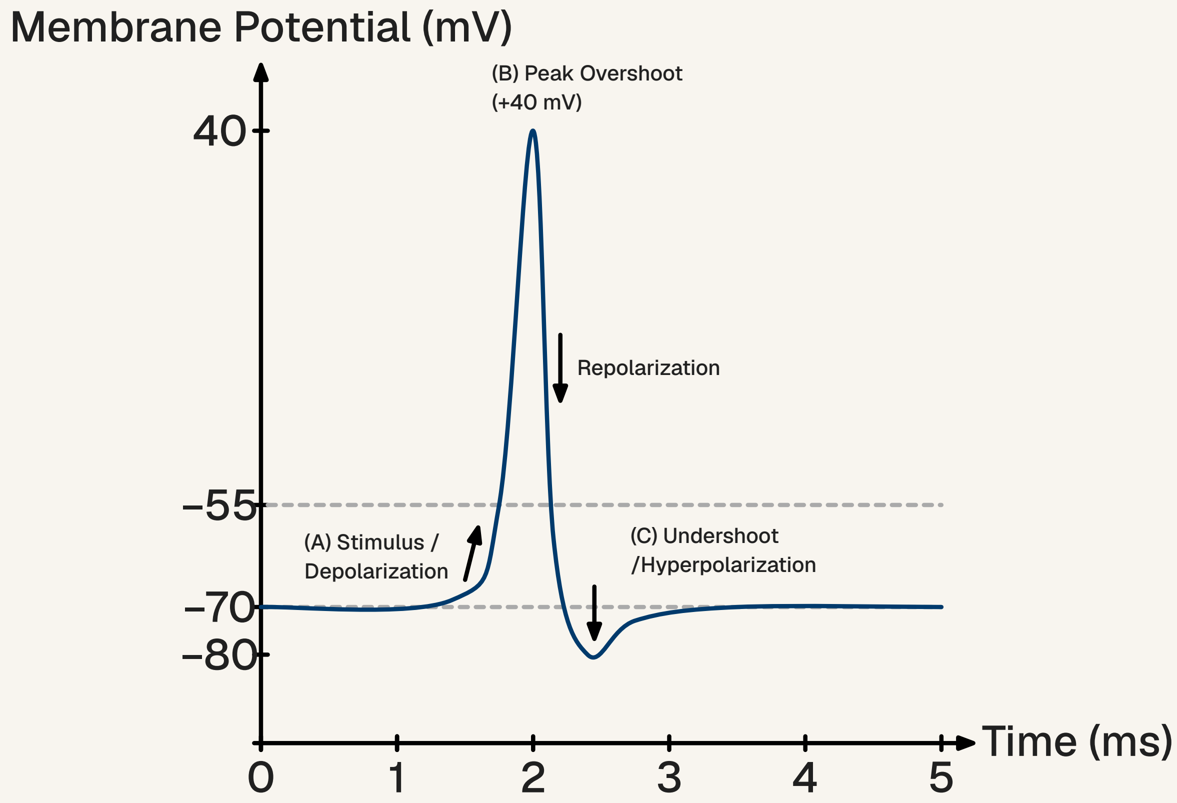
A biological neuron is a highly specialized cell that is the foundational building block in the nervous system. Functionally it is a event-driven processor with three functional zones:

- **Dendrites (Input):** Branching tree collecting signals from thousands of upstream neurons via synaptic terminals
- **Soma (Integration):** Cell body summing competing excitatory and inhibitory inputs; acts as a biological capacitor
- **Axon (Output):** Long cable transmitting the neuron's output spike to downstream targets; insulated by myelin for fast conduction

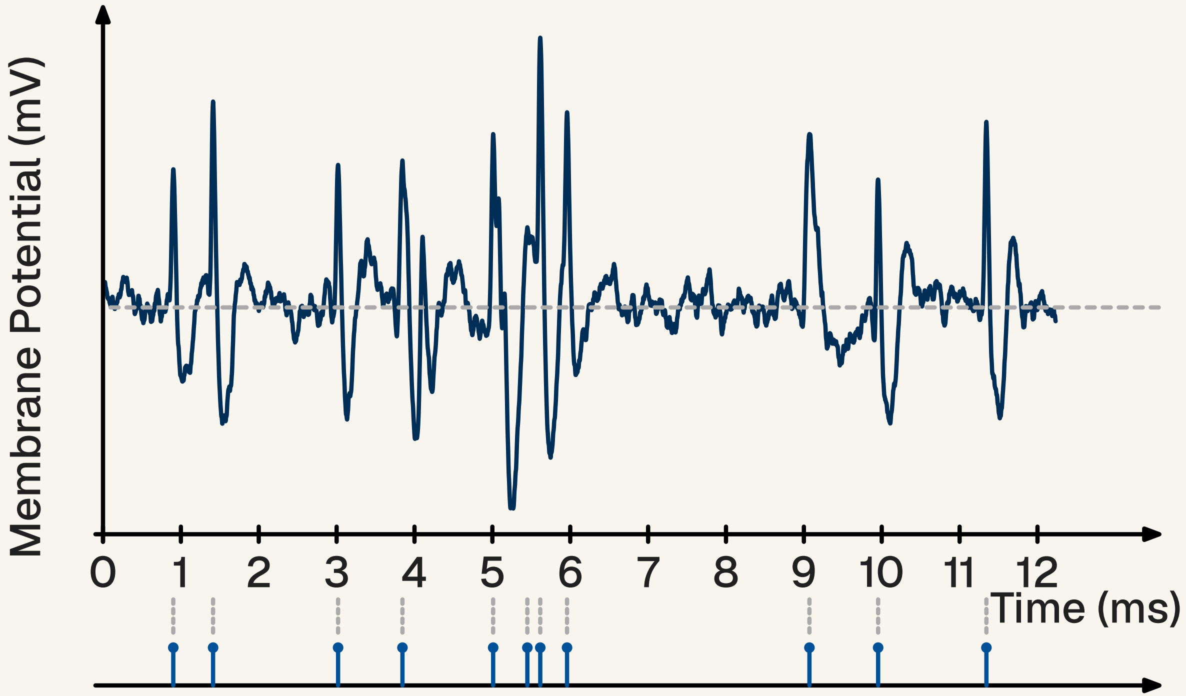
[9], [10]

All-or-nothing principle: If membrane potential exceeds the threshold, voltage-gated channels cascade open → rapid depolarisation spike. Sub-threshold noise is silently discarded. The neuron then enters a **refractory period**, resetting the gradient.

ACTION POTENTIAL & SPIKE TRAINS



Information is encoded entirely in the **precise timing** of spikes — abstracted as a sum of Dirac delta functions: $S(t) = \sum_f \delta(t - t_f)$



Because the spike waveform is stereotypical and invariant across neurons, **the waveform itself carries no information.** [10], [11]

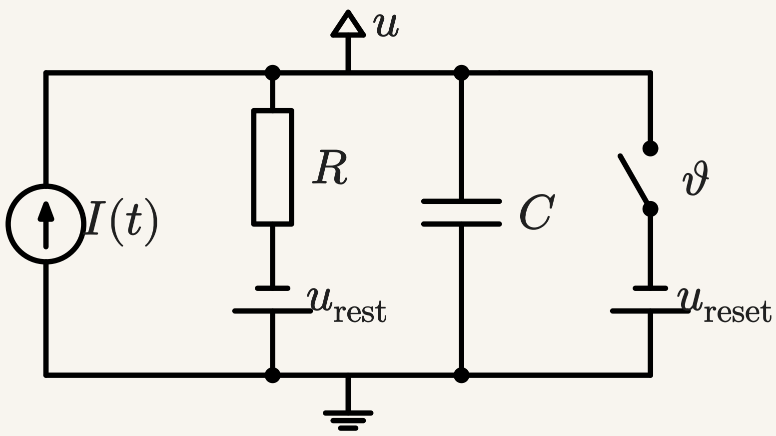
NEURON MODELS

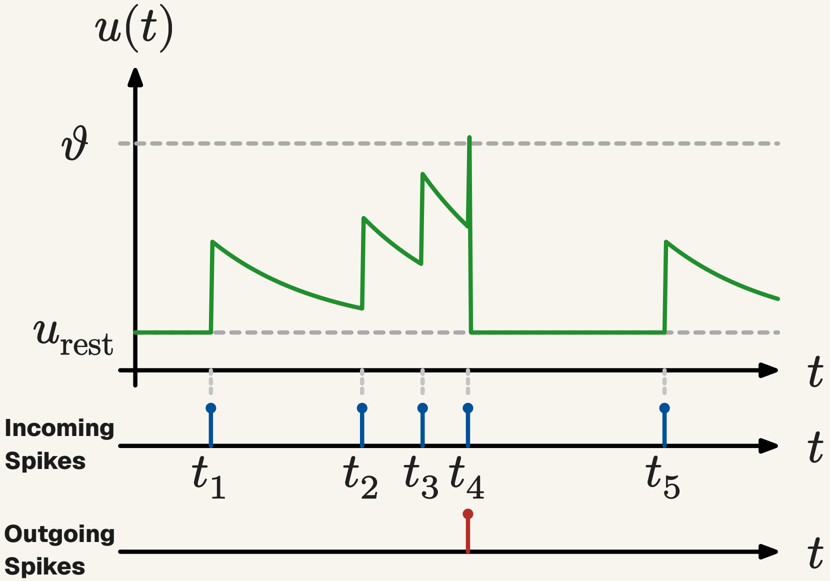
A fundamental trade-off: **biological realism** vs **computational efficiency**

- **Hodgkin-Huxley:** Full ionic channel dynamics — accurate, but requires supercomputing for large networks
- **Integrate-and-Fire (IF):** Pure arithmetic accumulator; fires when $u > \vartheta$, then resets. Minimal overhead — $O(1)$ per spike.
- **Leaky Integrate-and-Fire (LIF):** Adds exponential membrane decay τ_m:
- **Generalised LIF (GLIF):** Adds an adaptation variable $w(t)$ enabling bursting, frequency adaptation, and richer firing patterns

[12], [10]

LIF





The LIF forms the foundation of the other neuron models in this thesis. The main properties of the LIF are:

integration, firing when reaching threshold, leaking

ENCODING INFORMATION IN SPIKES

The brain occupies a unique middle ground between digital and analog:

- Spike output is **binary** — discrete, stereotypical events.
- Spikes can arrive and emit at any time, so it is **continuous in the time domain**.

This creates a **quantised continuous signal**: discrete amplitude, continuous time.

Requires a suitable codec to be useful as a communication medium.

Two primary coding schemes are hypothesised…

RATE ENCODING

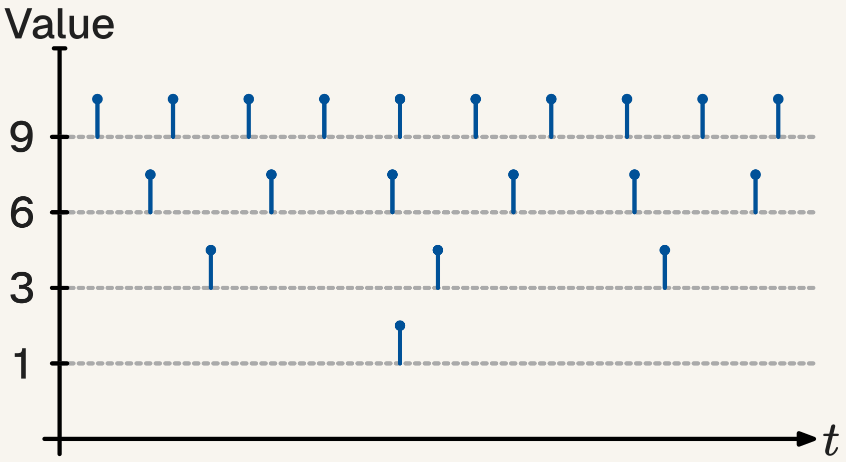
- Stimulus intensity is encoded as **mean firing frequency** over a time window — stronger stimulus → more spikes per second
- **Advantages:**
  - Simple and robust; directly observed in motor and sensory neurons
- **Limitations:**
  - latency barrier: The post-synaptic neuron must integrate spikes over tens–hundreds of milliseconds to estimate a reliable rate
  - This contradicts biological reaction times often under 100 ms, suggesting rate coding alone cannot account for time-critical processing [13]
  - On digital hardware, high firing rates cause rapid transistor switching — costly for power draw and bus congestion

[14], [10]

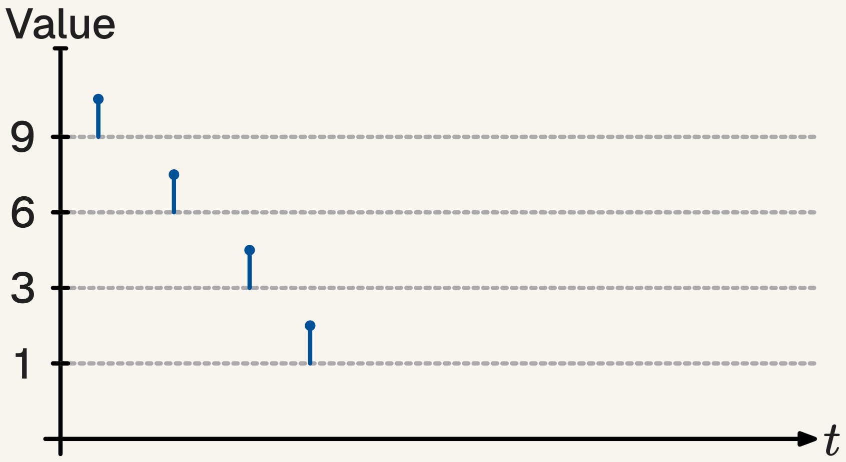
TIME-TO-FIRST-SPIKE (TTFS) ENCODING

- Unlike Rate Coding, which requires extended time windows, TTFS encodes stimulus intensity inversely to response latency
- **Mechanism:** A high-intensity (bright) pixel triggers an early spike, compressing spatial information into a priority-driven queue
- **Advantages:**
  - **Sparsity:** Sub-threshold noise (background pixels) is aggressively discarded and never fires
  - **Early Exit:** Salient features arrive first, enabling confident decisions within the first 15% of the temporal window
  - **Latency:** Eliminates the need to wait for a time window to close; processing terminates as soon as the threshold is reached
- **Limitations:**
  - **Phase ambiguity** – cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one without a reference signal.
  - **Sensitive to noise**

[15], [10]

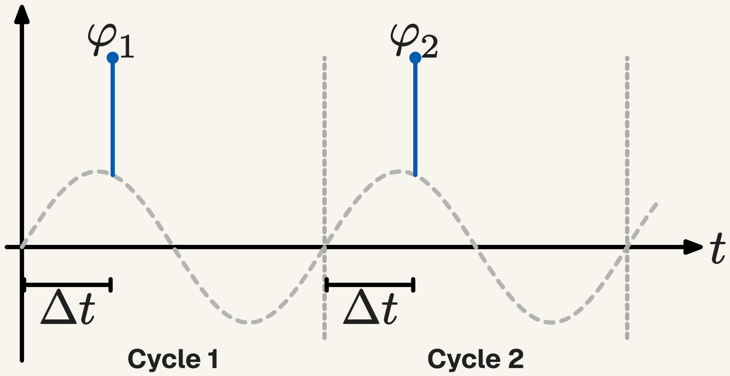


Rate encoded



TTFS encoded

TTFS PHASE AMBIGUITY



$$\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$$

Indistinguishable without a cycle counter

Spikes occurring at the same relative phase (φ_1 and φ_2) across different oscillation cycles are mathematically indistinguishable ($\varphi_1 = \varphi_2 (\mathrm{mod}\, 2\pi)$). Without a mechanism to track the global cycle count, downstream neurons cannot determine whether a spike represents a delayed response to a previous stimulus or an early response to a new one.

BIOLOGICAL NEURAL NETWORKS

Lateral Inhibition and Winner-Takes-All (WTA):

- An active excitatory neuron stimulates nearby inhibitory neurons, which in turn suppress competing excitatory neighbours
- This creates a **WTA dynamic** — the most active neuron silences its competitors, providing a physical mechanism for categorical decisions without a central processor

Excitation-Inhibition balance is critical:

- Excess excitation → runaway feedback loops (analogous to seizures)
- Excess inhibition → signal extinction (quiescence)

The brain operates at the critical point between these two extremes. Achieving this balance is essential in engineered SNNs.

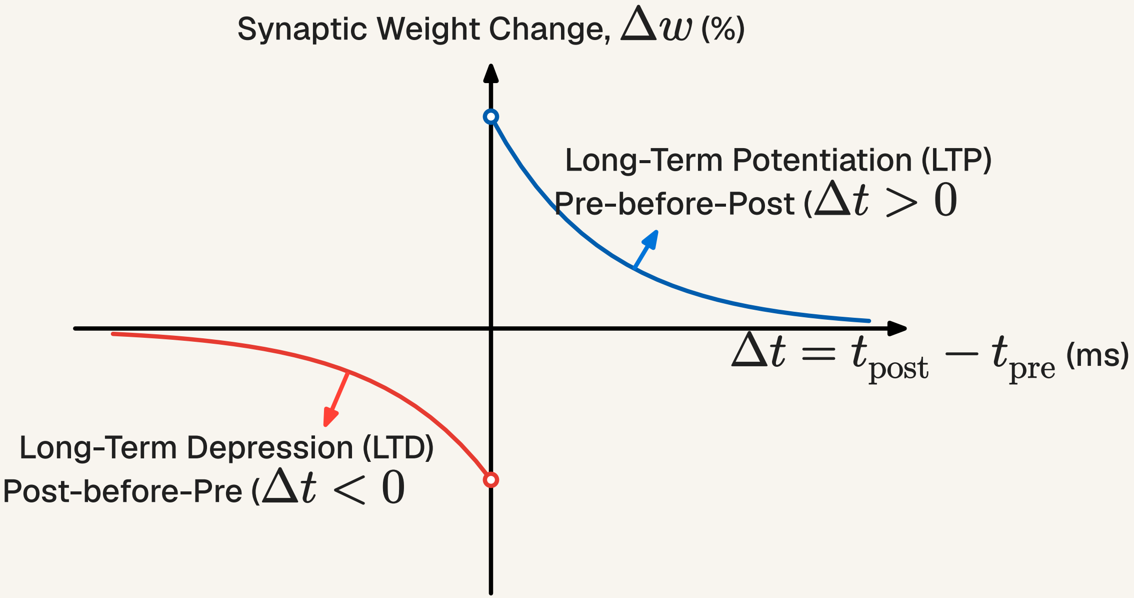
BIOLOGICAL LEARNING

Spike-Timing-Dependent Plasticity (STDP):

A causal, millisecond-scale refinement of Hebbian learning. Weight update depends on the timing difference $\Delta t = t_{\text{post}} - t_{\text{pre}}$:

- **Long-Term Potentiation (LTP):** Pre fires **before** post ($\Delta t > 0$) — input contributed to firing → synapse **strengthened**
- **Long-Term Depression (LTD):** Pre fires **after** post ($\Delta t < 0$) — input was irrelevant → synapse **weakened**

- Fully local: no global error signal required — only pre- and post-synaptic activity at the cleft
- **Homeostatic plasticity** prevents runaway growth: if a neuron's average rate exceeds a target, all incoming weights are globally scaled down


Synaptic Weight Change, Δw (%)
Long-Term Potentiation (LTP)
Pre-before-Post ($\Delta t > 0$
$\Delta t = t_{\text{post}} - t_{\text{pre}}$ (ms)
Long-Term Depression (LTD)
Post-before-Pre ($\Delta t < 0$

BACK TO WHY DEEP LEARNING IS INEFFICIENT

Deep learning shares many core ideas with neuroscience but has diverged into its own discipline over time.

SIMILARITIES

- Deep learning has a version of neurons (the perceptron); it integrates and fires when reaching a threshold.
- It has neural networks—neurons connected together in hierarchical topologies to extract features.

DIFFERENCES

- **Signals:** ANNs use continuous, synchronous floating-point values (activations). The brain uses discrete, asynchronous events (spikes).
- **Optimization:** ANNs rely on backpropagation (global error gradients and exact weight transport). The brain relies on local plasticity (STDP).
- **Hardware Mapping:** ANNs execute dense matrix multiplications regardless of data sparsity. Biological networks communicate sparsely, saving massive amounts of energy.

BOTTLENECKS OF DEEP LEARNING

Standard deep learning imposes four compounding hardware bottlenecks:

- **Von Neumann Bottleneck:** Weights shuttle between memory and compute each step. Moving 32 bits from DRAM costs 640 pJ; computing with them costs 0.1 pJ — **Energy is wasted on transport, not computation**
- **Dense Processing of Sparse Data:** GPUs execute $0 \times w = 0$ multiplications for zero activations — structurally blind to sparsity, even when ReLU produces mostly-zero activation maps
- **Clock Synchrony Tax:** The global clock distribution network alone consumes 30–40% of chip power, even when circuits are idle
- **Backpropagation Locking:** All intermediate activations must remain in VRAM for the full backward pass; local synapses cannot adapt until the global error loop completes

NEUROMORPHIC PRINCIPLES

Three architectural pillars that directly address each bottleneck:

Co-location of Memory and Compute: Synaptic weights live with the neuron — zero data transport cost. Learning and inference are co-located.

Event-Driven Asynchrony: No global clock. Energy scales with task activity, not network size. Silent neurons consume nothing.

Sparse Binary Communication: Information is encoded in spike timing, not magnitude. Bandwidth scales with information content, not data dimensionality.

Local learning rules: Synapses can update independent of eachother avoiding the backpropagation lock.

METHOD

To answer the research questions:

Sparse Efficient Computing: Investigate whether biologically inspired, event-driven algorithms reduce the computational footprint of visual classification when simulated on standard hardware.

Neuron Model Evaluation: Identify which temporal integration dynamics are compatible with Time-To-First-Spike (TTFS) rank-order decoding across systematic threshold regimes.

Inference Via Weight Transfer: Quantify the accuracy penalty of zero-shot Artificial-Neural-Network (ANN) to Spiking-Neural-Network (SNN) weight transfer under TTFS encoding, isolating the cost of transitioning to event-driven integration.

Native Unsupervised Learning: Determine if local Spike-Time-Dependent-Plasticity (STDP) can autonomously extract meaningful geometric features from visual input without global error signals.

We propose the following experimental setup…

EXPERIMENT SETUP

The experiments are divided into three phases:

- **Phase I:** Neuron model evaluation — finding a suitable and efficient neuron model to use for Phase II and Phase III.
- **Phase II:** Weight transfer — using a pre-trained network via established methods and transferring weights to an SNN to investigate computational savings.
- **Phase III:** Training an SNN from scratch with zero dependence on traditional frameworks for potentially maximum efficiency.

DATASET

MNIST (28×28 normalized grayscale images), chosen for its simplicity and high degree of spatial sparsity.
Need to encode image as TTFS spike train ($t_i = (1 - p_i) \cdot 64, p_i \in [0, 1]$, background suppressed)

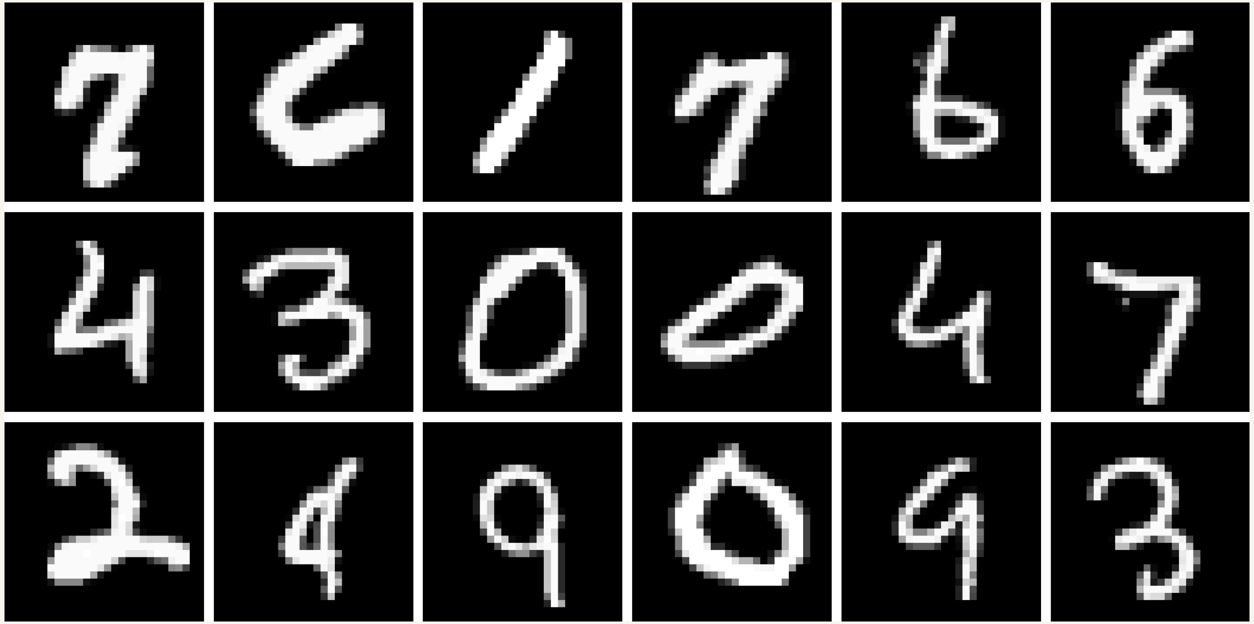
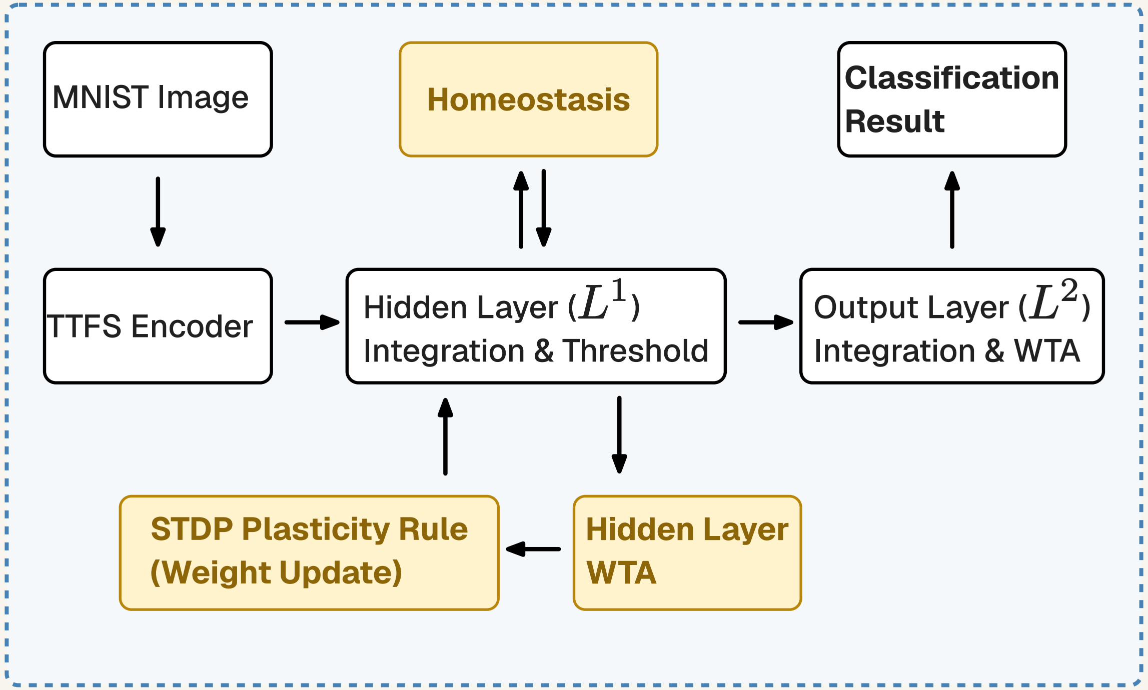


Figure 1: Sample of the MNIST dataset. The 28x28 images are normalized and flattened into 1D vectors before being translated into temporal spike events.

NETWORK ARCHITECTURE

- **Topology:** Fully Connected Network (FCN) $784 \to 128 \to 10$
  - An FCN allows mathematically transparent, one-to-one parameter mapping without the structural overhead of convolutional unrolling
  - **No bias terms** — to preserve pure event-driven sparsity and simplify ANN→SNN transfer
- **SNN Simulator:** Custom PyTorch discrete-time engine operating over a $T_{\max} = 64$ tick saccade window (arbitrary choice)
- **WTA:** Winner-Takes-All (WTA) — first output neuron to cross threshold claims the class label WTA is also used in the hidden layer durin learning with STDP


MNIST Image
Homeostasis
Classification Result
TTFS Encoder
Hidden Layer (L^1) Integration & Threshold
Output Layer (L^2) Integration & WTA
STDP Plasticity Rule (Weight Update)
Hidden Layer WTA

PHASE I: NEURON MODEL EVALUATION

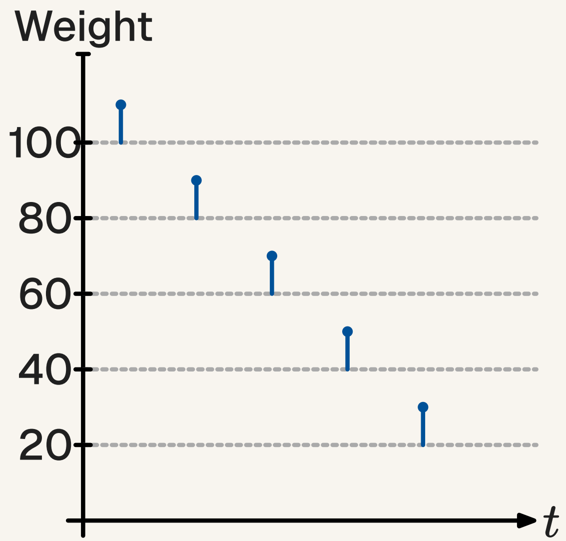
A **unit test** for isolated neuron temporal dynamics. Two synthetic spike trains are constructed:

- **Concordant:** Strongest synaptic weights arrive **first** — aligned with TTFS rank-order priority
- **Discordant:** Strongest weights arrive **last** — inverted temporal order

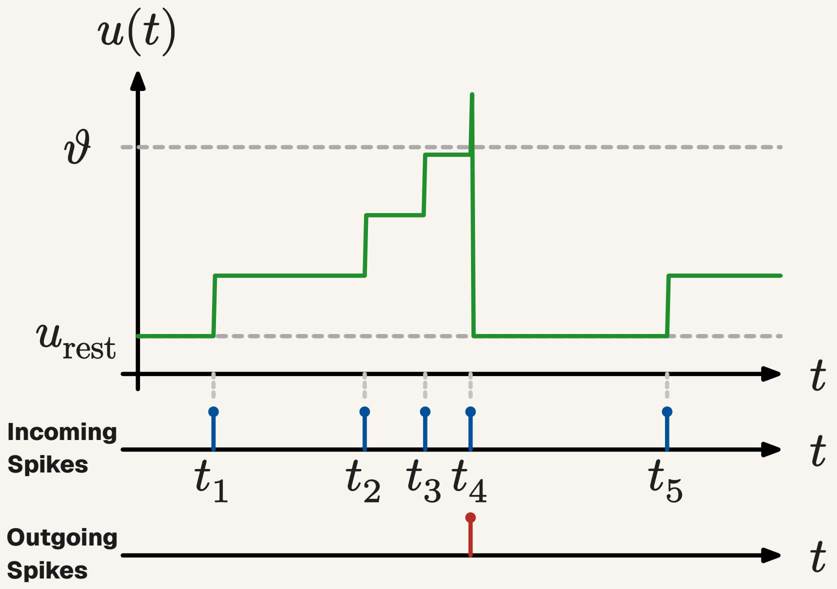
Each model is then evaluated across three threshold regimes:

- **Saturation (low ϑ):** Does the model fire early when evidence is abundant?
- **Critical (balanced ϑ):** Can the model discriminate **temporal order**, not just total magnitude?
- **Deficit (high ϑ):** Does the model fail gracefully when total weight is insufficient to cross the threshold?

No MNIST data is used — this phase isolates the neuron's mathematical response to controlled spike patterns.

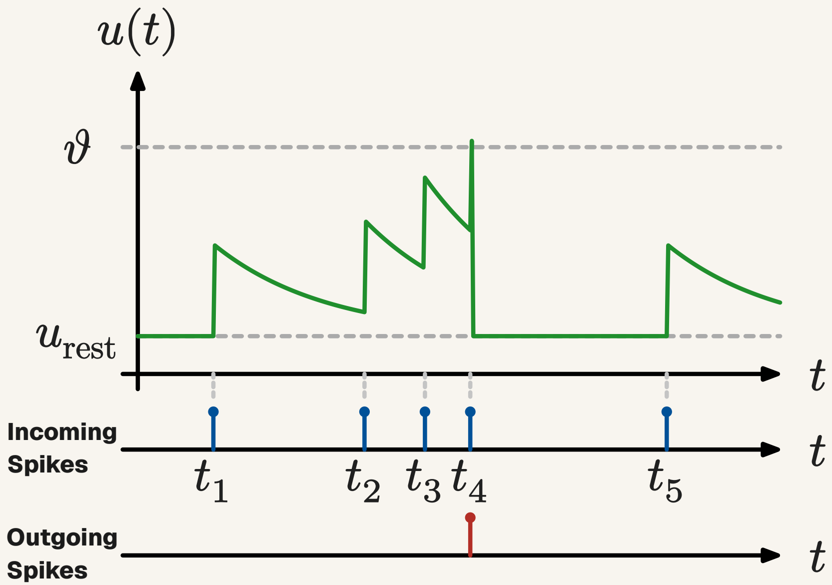

Weight
100
80
60
40
20
t
Weight
20
40
60
80
100
t

IF MODEL (A)

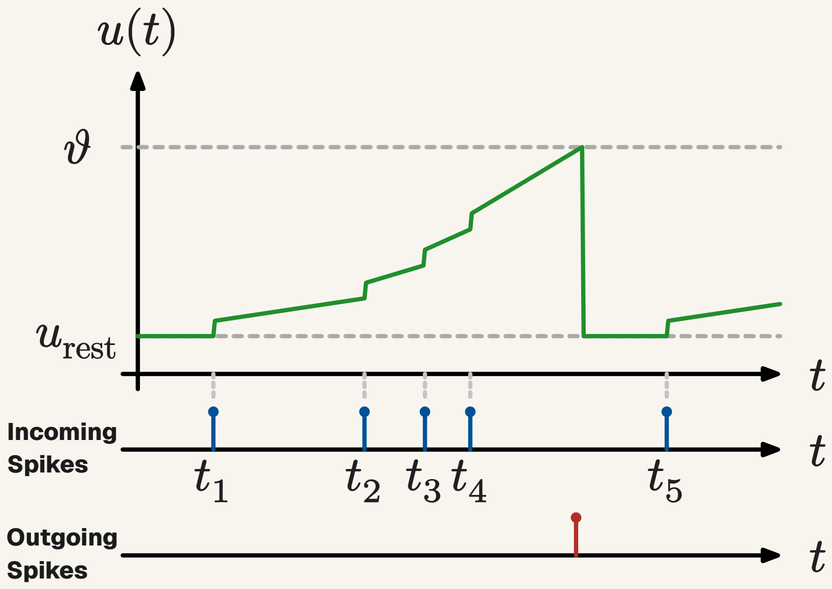


$u(t)$
ϑ
u_{rest}
t
Incoming Spikes
t_1
t_2
t_3
t_4
t_5
t
Outgoing Spikes
t

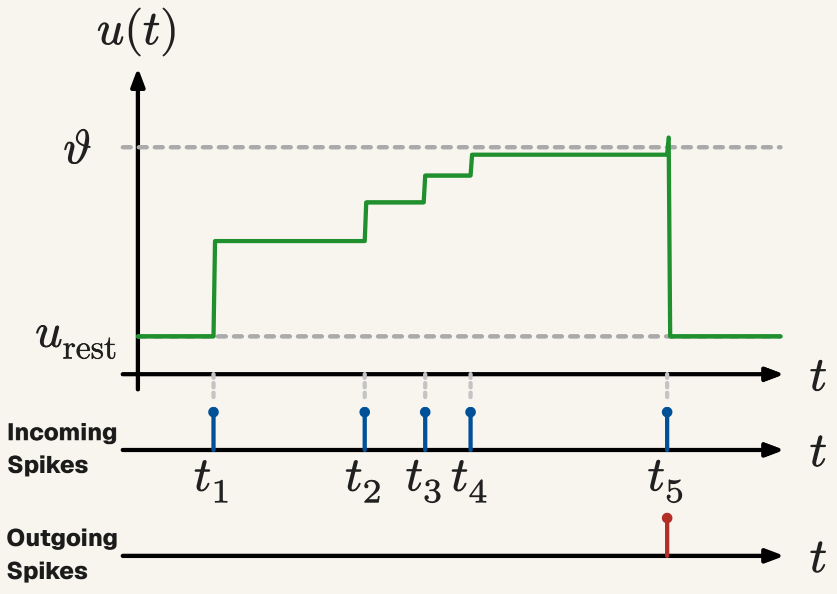
LIF MODEL (B)


$u(t)$
ϑ
u_{rest}
t
Incoming
Spikes
t
t_1
t_2
t_3
t_4
t_5
Outgoing
Spikes
t

LINEAR ACUMULATING RAMP MODEL (C)



STATE DISCOUNT MODEL (D)



PHASE II: ZERO-SHOT WEIGHT TRANSFER

1. Train a standard ANN baseline ($784 \to 128 \to 10$, Adam optimizer, 1000 epochs, no bias terms)
2. Copy FP32 weights directly to a structurally identical SNN — scaled by $\times\, 64$ to fit with the neuruon models (arbitrary choice)
3. Run SNN inference using Model C over $T_{\max} = 64$ ticks with WTA output

Goal: Measure the accuracy lost purely by switching from static continuous dot-product activations to discrete TTFS momentum integration, with **no retraining**.

Efficiency is tracked via SyOPs vs MACs and mean time-to-decision latency across the 10,000-image test set.

PHASE III: LEARNING

The SNN is trained **from scratch** — randomized weights, no labels, no backpropagation. Learning loop per image (one saccade):

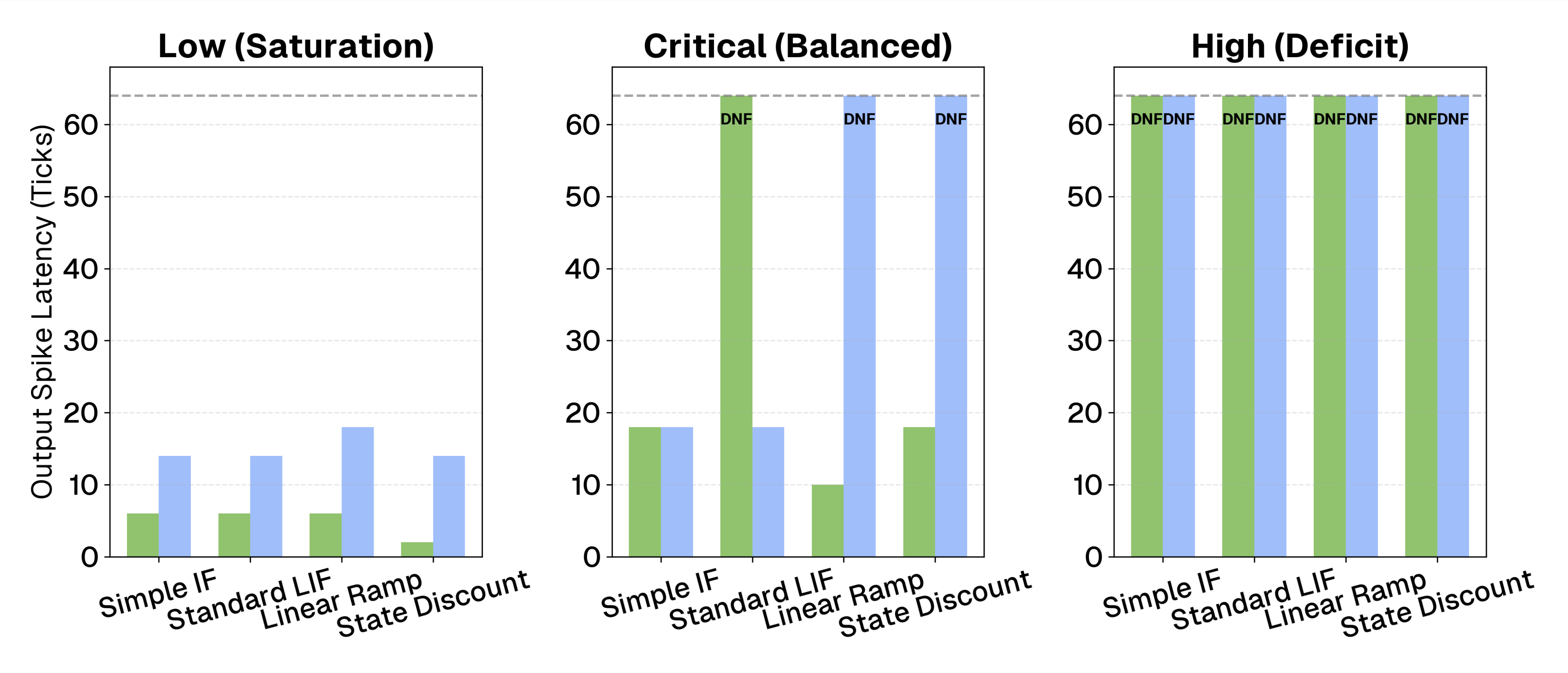
1. Forward pass — all 128 hidden neurons integrate independently via Model C
2. **Post-hoc Hard WTA:** Only the single earliest-firing neuron is permitted to update
3. **Vectorized STDP:** Pre-spikes before the winner's time → LTP ($+A_+$); after or silent → LTD ($-A_-$)
4. **Homeostatic adaptation:** Winner receives threshold penalty ($+600$); all thresholds decay ($\times\ 0.90$)

> Post-hoc labeling after training: pass labeled data through the frozen network; assign each output neuron the digit class it most frequently wins for. Then measure accuracy on the test set.

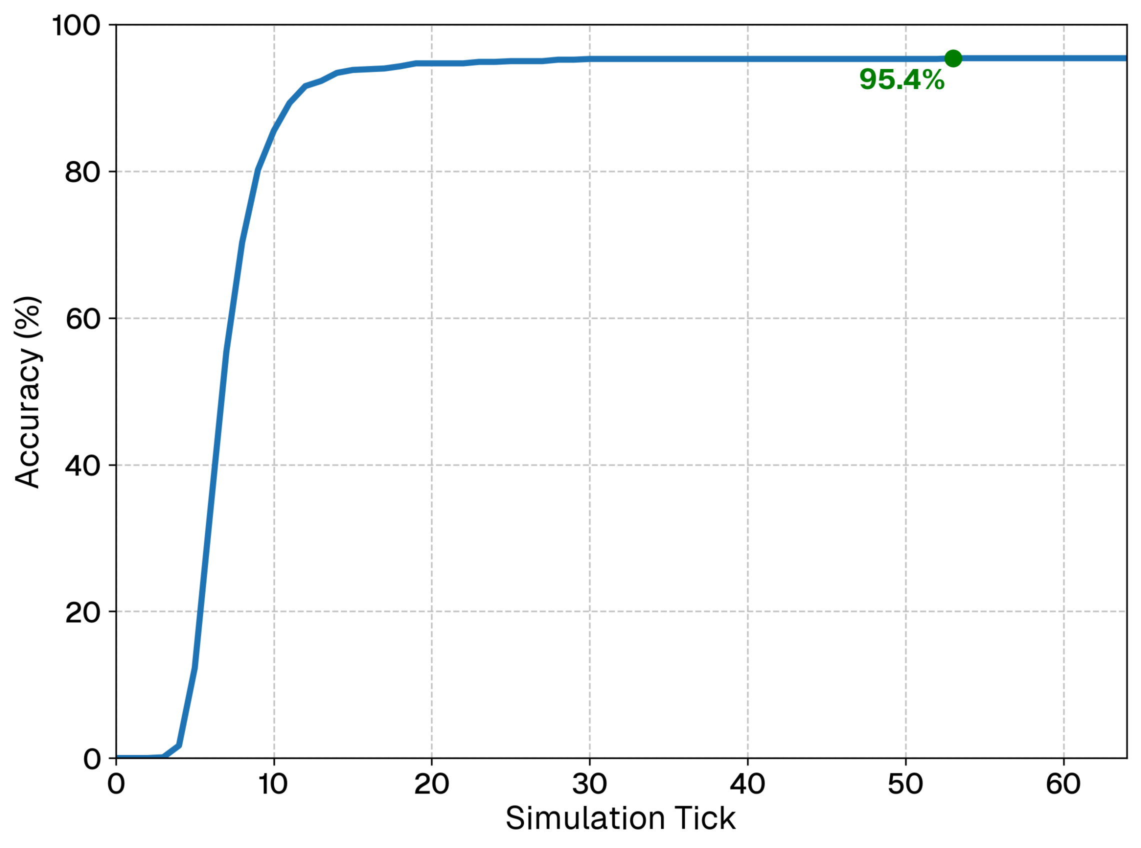
METRICS

- Top-1 Accuracy
- Temporal Latency (Time-to-Decision)
- Synaptic Operations (SyOPs) as a hardware proxy

PHASE I: NEURON MODEL EVALUATION – RESULTS



PHASE II: ZERO-SHOT WEIGHT TRANSFER – RESULTS



Model Configuration	Accuracy (%)
ANN (Static FP32, Bias=False)	98.40%
SNN (Spiking Model C, FP32)	94.50%

Table 1: Performance degradation breakdown. The Temporal Penalty isolates the loss of TTFS encoding without any weight alteration besides scaling.

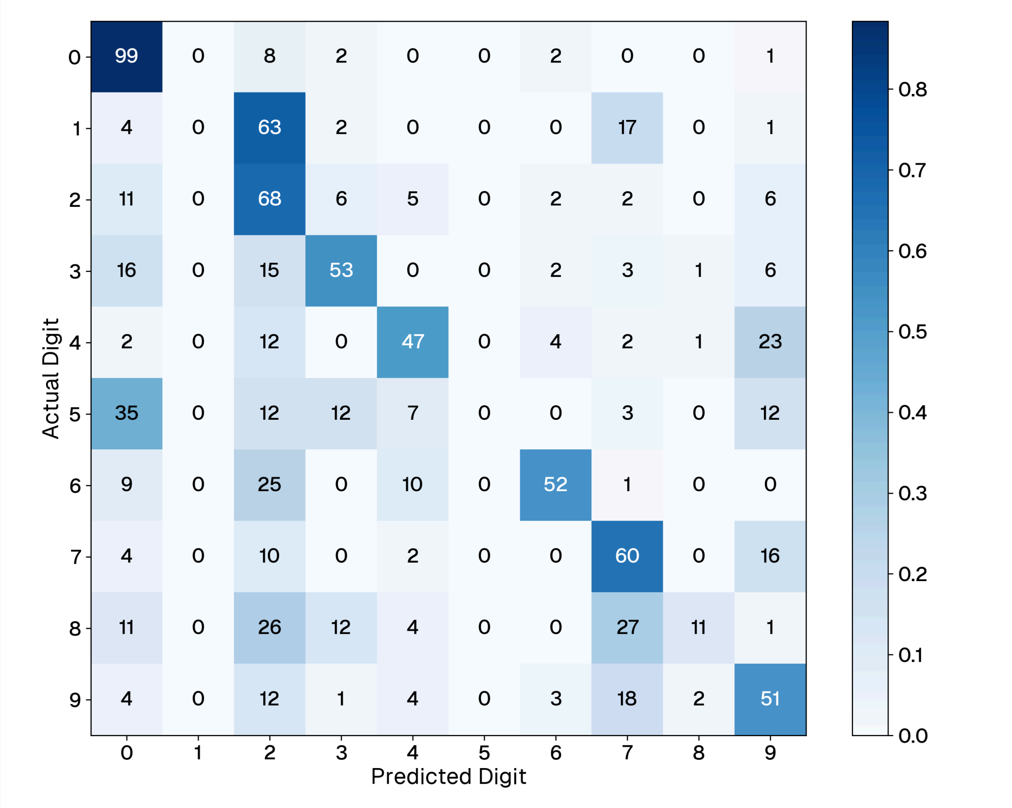
Architecture	Metric Target	Avg. Latency	Avg. Operations / Image	Compute Reduction
ANN (Static FP32)	Dense MACs	N/A (Static)	101,632 MACs	0%
SNN (Spiking FP32)	Sparse SynOps	8.4 Ticks	15,021 SynOps	85.2%

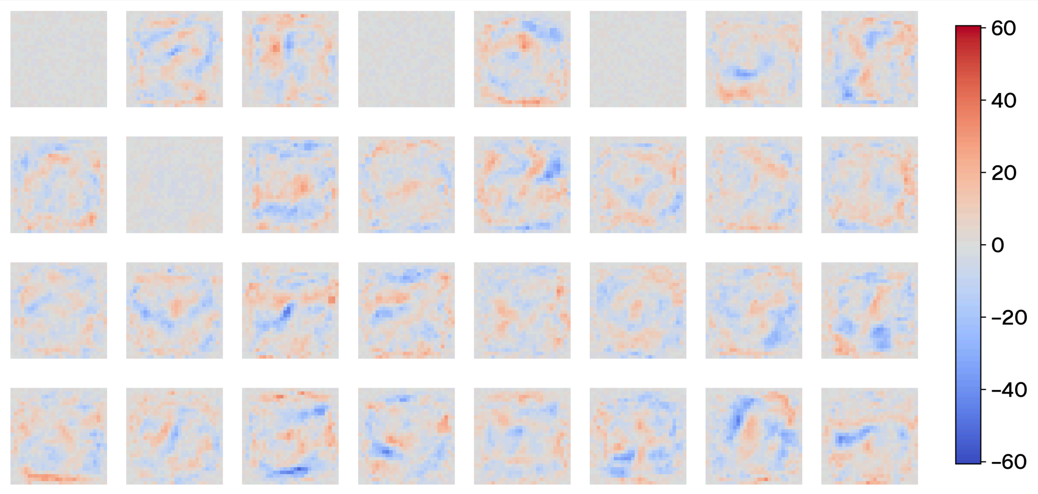
Table 2: Computational cost comparison. The SNN significantly reduces operations by leveraging temporal early-exit sparsity.

The S-curve profile confirms the **early-exit hypothesis**: >80% of correct decisions are locked in between ticks 5–15. Mean time-to-decision: **8.4 ticks** out of 64 → **85.2% reduction** in operations (15,021 SyOPs vs 101,632 MACs).

PHASE III: LOCAL SNN LEARNING – RESULTS

Unsupervised STDP + Hard WTA + homeostasis: **44.3% top-1 accuracy**





,

- Mean latency: **8.3 ticks** — comparable to supervised transfer
- SyOPs: **12,954 → 87.2% reduction** (marginally sparser than Phase II)

Hidden neurons self-organized into holistic digit templates without any labels. Visually simple digits (‘0’, ‘1’) cluster reliably; morphologically ambiguous pairs (‘5’/‘0’, ‘4’/‘9’) cause the most confusion.

DISCUSSION: KEY FINDINGS & LIMITATIONS

Neuron Model Evaluation:

- Classical LIF may be misaligned with TTFS — biological fidelity must be selectively adapted for engineering objectives
- Model C achieves temporal discriminability through purely linear mechanics, avoiding expensive exponential kernels

Weight Transfer Viability:

- 94.5% zero-shot accuracy proves ANN spatial hierarchies are recoverable from spike timing alone — hybrid pipelines are viable for edge deployment
- **Sparsity Paradox:** GPUs still execute boolean-masked zero multiplications; physical efficiency gains require native neuromorphic ASICs or FPGAs or smarter GPU implementations

Unsupervised Learning:

- STDP clusters geometric features without labels — the 44.3% result is not a failure of the learning rule but of the single-layer fully-connected WTA topology
- Holistic template matching breaks down on morphologically overlapping digit classes

FUTURE WORK

- **Complex datasets:** Imagenet, N-MNIST, DVS Gesture sensors — output spike trains natively, need to handle TTFS contrast
- **Spiking CNN architectures:** Local receptive fields + local WTA → neurons learn edge and curve primitives rather than rigid full-digit templates, enabling translation invariance
- **Structural plasticity:** Prune consistently depressed synapses → block-sparse tensor formats → real memory bandwidth reductions on FPGAs and ASICs
- **Hardware deployment:** Port verified TTFS algorithms to Intel Loihi, IBM TrueNorth or custom accelerators to measure true joule-per-inference energy consumption

CONCLUSION

The brain runs on **20 W**. Modern AI clusters run on **megawatts**. Closing this gap is a paradigmatic challenge.

This thesis demonstrated:

- **TTFS encoding** reduces theoretical compute by 85–87% vs. dense ANN inference via temporal early-exit sparsity
- **Standard LIF may be incompatible with TTFS** — Model C (current-accumulating linear ramp) is a suitable model if we need to decode rank order
- **Zero-shot weight transfer** recovers 94.5% accuracy with only a 3.9 pp temporal penalty — no retraining required
- **Unsupervised STDP** successfully extracts geometric features (44.3%) without labels or backpropagation

GPUs still perform masked zero multiplications: **realizing these efficiency gains requires native neuromorphic substrates** or smarter GPU implementations.

The majority of the teoretical savings come from the TTFS encoding and the challange is building systems around this.

BIBLIOGRAPHY

[1] J. Jumper *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, no. 7873, pp. 583–589, Aug. 2021, doi: 10.1038/s41586-021-03819-2.

[2] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016, doi: 10.1038/nature16961.

[3] E. Strubell, A. Ganesh, and A. McCallum, “Energy and Policy Considerations for Deep Learning in NLP,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Florence, Italy: Association for Computational Linguistics, 2019, pp. 3645–3650. doi: 10.18653/v1/P19-1355.

[4] R. Geirhos *et al.*, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, Nov. 2020, doi: 10.1038/s42256-020-00257-z.

[5] M. Horowitz, “1.1 Computing's energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, San Francisco, CA, USA: IEEE, Feb. 2014, pp. 10–14. doi: 10.1109/ISSCC.2014.6757323.

[6] S. J. Martin, P. D. Grimwood, and R. G. M. Morris, “Synaptic Plasticity and Memory: An Evaluation of the Hypothesis,” *Annual Review of Neuroscience*, vol. 23, no. 1, pp. 649–711, Mar. 2000, doi: 10.1146/annurev.neuro.23.1.649.

[7] E. Friedman, “Clock distribution networks in synchronous digital integrated circuits,” *Proceedings of the IEEE*, vol. 89, no. 5, pp. 665–692, May 2001, doi: 10.1109/5.929649.

[8] S. Laughlin, “Energy as a constraint on the coding and processing of sensory information,” *Current Opinion in Neurobiology*, vol. 11, no. 4, pp. 475–480, Aug. 2001, doi: 10.1016/S0959-4388(00)00237-3.

[9] E. R. Kandel, J. Koester, S. Mack, and S. Siegelbaum, Eds., “Principles of neural science.” McGraw Hill, New York, 2021.

[10] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal dynamics: from single neurons to networks and models of cognition*. Cambridge, United Kingdom: Cambridge University Press, 2014.

[11] B. Metcalfe, “Action potentials recorded from the L5 dorsal rootlet of rat using a multiple electrode array.” Accessed: Jan. 08, 2026. [Online]. Available: https://data.mendeley.com/datasets/ybhwtngzmm/1

[12] A. L. Hodgkin and A. F. Huxley, "A quantitative description of membrane current and its application to conduction and excitation in nerve," *The Journal of Physiology*, vol. 117, no. 4, pp. 500–544, Aug. 1952, doi: 10.1113/jphysiol.1952.sp004764.

[13] S. Thorpe, D. Fize, and C. Marlot, "Speed of processing in the human visual system," *Nature*, vol. 381, no. 6582, pp. 520–522, June 1996, doi: 10.1038/381520a0.

[14] E. D. Adrian and D. W. Bronk, "The discharge of impulses in motor nerve fibres: Part II. The frequency of discharge in reflex and voluntary contractions," *The Journal of Physiology*, vol. 67, no. 2, pp. 9–151, Mar. 1929, doi: 10.1113/jphysiol.1929.sp002557.

[15] R. V. Rullen and S. J. Thorpe, "Rate Coding Versus Temporal Order Coding: What the Retinal Ganglion Cells Tell the Visual Cortex," *Neural Computation*, vol. 13, no. 6, pp. 1255–1283, June 2001, doi: 10.1162/08997660152002852.